

水样图像 在线检测系统

你提供的水色图片，我们可以给出相应的分析，让你更轻松地了解水质的好坏。

更多



基于水色图像的水质评价

教学目标与基本要求:

教学目标:

基于水色图像进行水质评价的案例，重点介绍了图像处理算法中的颜色矩提取和数据挖掘算法中决策树算法在实际案例中的应用。利用水色图像颜色矩的特征，采用决策树算法进行水质评价，并详细的描述了数据挖掘的整个过程。

基本要求:

1. 掌握图像处理算法中的颜色矩提取。
2. 掌握决策树算法的使用。
3. 了解决策树算法在水质评价实例中的应用。

问题背景:

水产养殖，人类生活，生物生长都离不开水，而不同的用水需求对水质要求不同。

- 虽然有些专家和有经验的农民可以直接通过观察水色来确定水质的好坏，但测出来水质还存在主观性的偏差。
- 现在科学技术已经发明了一些水质检测仪器，但用仪器检测水质费时费力，无法满足没有仪器却需要了解水质的情况。
- 通过计算机建模对多种水色图像提取特征与分析，从而判断水质的好坏显得尤为迫切。

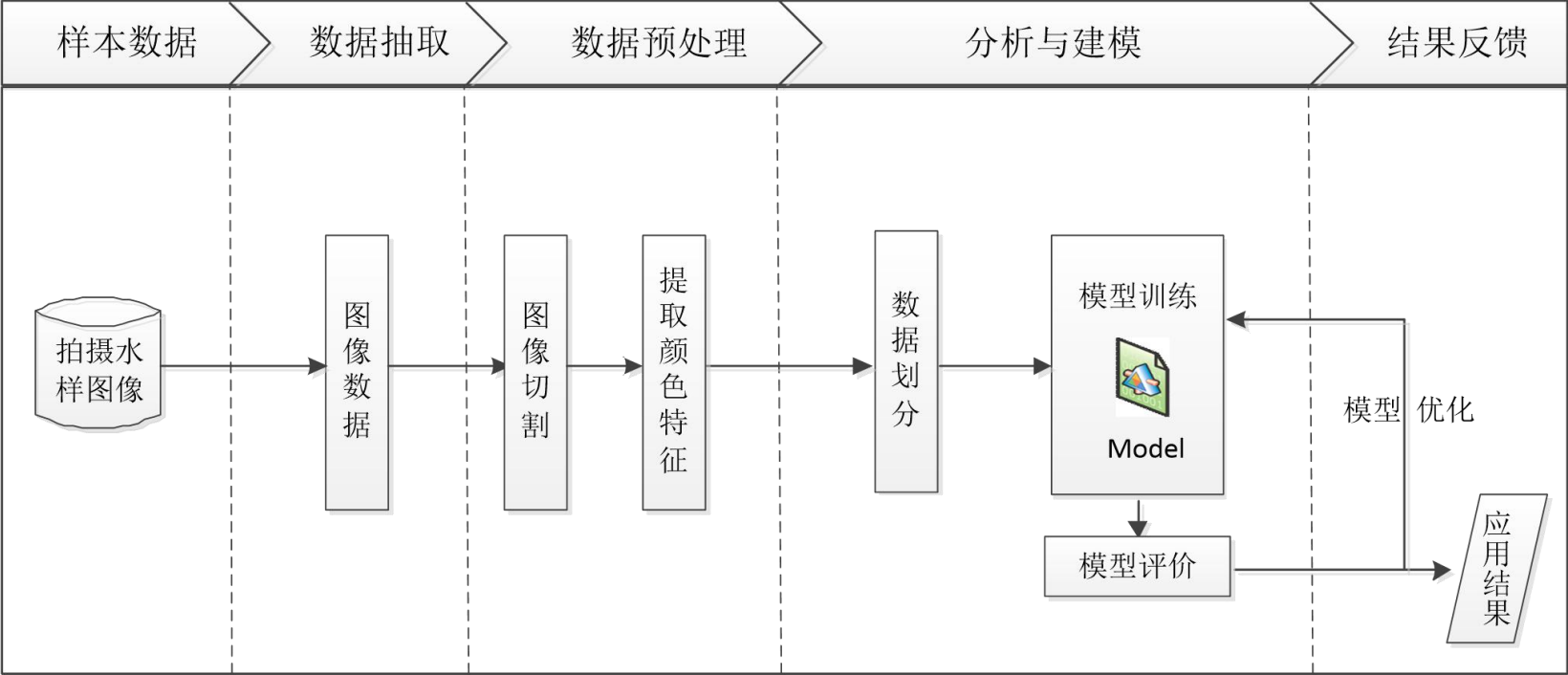
分析方法与过程

基于水色图像特征提取的水质评价步骤如下。

1. 从采集到的原始水样图像中进行选择性抽取形成建模数据。
2. 对步骤1形成的数据集进行数据预处理，包括图像切割和颜色矩特征提取。
3. 利用步骤2形成的已完成数据预处理的建模数据，划分为训练集与测试集。
4. 利用步骤3的训练集构建分类模型。
5. 利用步骤4的构建好的分类模型进行水质评价。

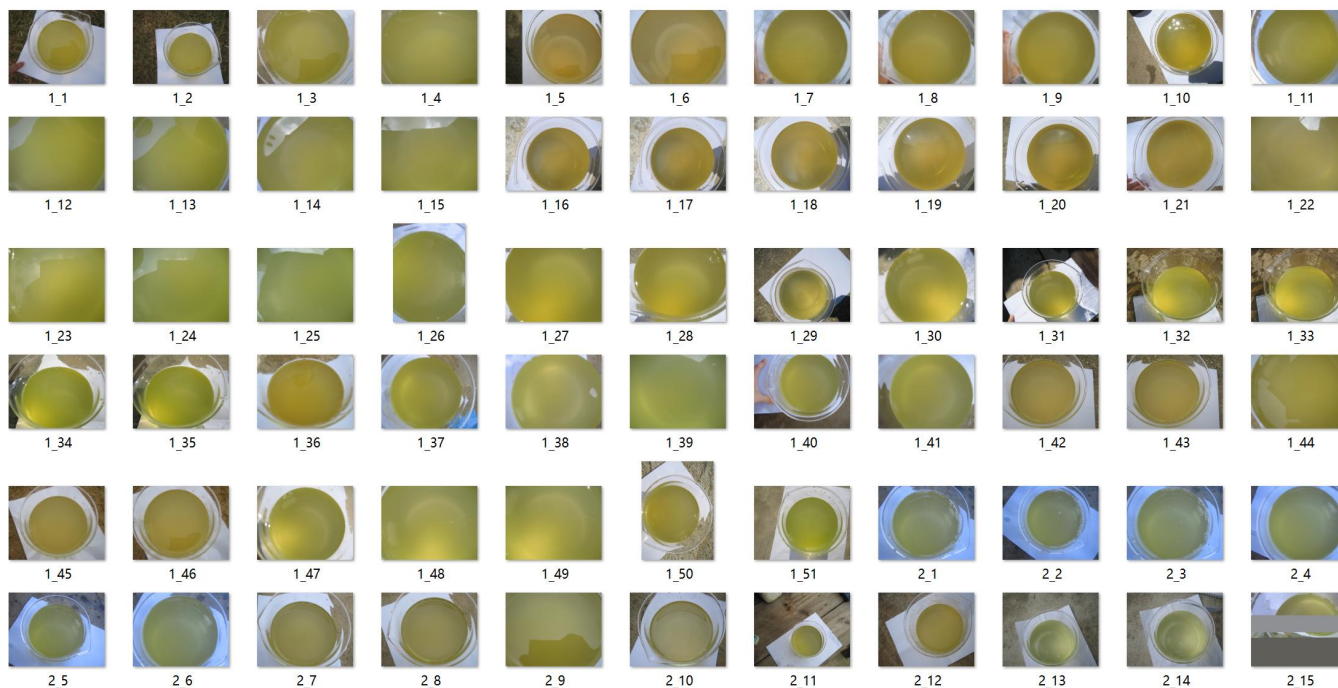
分析方法与过程

基于水色图像特征提取的水质评价流程图如下。



水色分类

每个水质图片命名规则为“类别-编号.jpg”，如“1_1.jpg”说明当前图片属于第1类的样本。



水色	浅绿色 (清水或 浊水)	灰蓝色	黄褐色	茶褐色 (姜黄、茶褐、红褐、褐中带绿 等)	绿色(黄绿、油绿、蓝绿、 墨绿、绿中带褐等)
水质类别	1	2	3	4	5

水色分类

根据专家鉴别出来的五种水质类别，将五种水色类别浅绿色标记为“1”，灰蓝色为“2”，黄褐色为“3”，茶褐色为“4”，绿色为“5”，之后数据集中训练集和测试集表示时，用“1”“2”“3”“4”“5”来表示分类后的水质类别。



黄褐色

以硅藻，门藻类为主，常伴有少量的裸藻。



黄绿色，淡绿色

好水，水中水质肥度程度适中



浓蓝绿色

有较大颗粒的藻类，甚至出现藻团，为坏水。



黑褐色

池水出现了泥土一样的颜色，早黑晚浓。



红棕色

水体类角类大量繁殖，水体缺氧。

```
import numpy as np
import os, re
from PIL import Image
```

图像切割及特征提取

```
DEFAULT_PATH = '../data/images/' # 图片所在路径
def getImgNames(path= DEFAULT_PATH): # 自定义获取图片名称函数
```

"""

获取指定路径中所有图片的名称

:param path: 指定的路径

:return: 名称列表

"""

```
filenames = os.listdir(path)
```

```
imgNames = []
```

```
for i in filenames:
```

```
    if re.findall(r'^\d_\d+\.jpg$', i) != []:
```

```
        imgNames.append(i)
```

```
return imgNames
```

批量处理图片数据

imgNames = getImgNames() # 获取所有图片名称

n = len(imgNames) # 图片张数

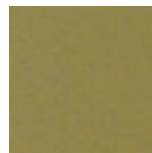
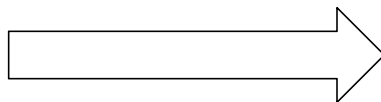
print(n)

203 张
图片

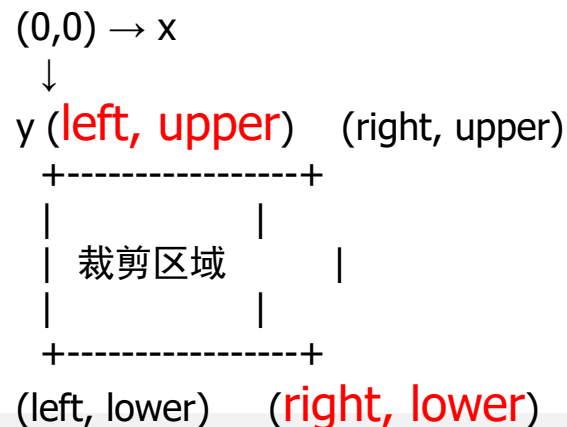
分析方法与过程

第一步：图像切割

- 采集到的水样图像包含盛水容器，容器的颜色与水体颜色差异较大，同时水体位于图像中央，为了提取水色的特征，需要提取水样图像中央部分具有代表意义的图像，具体实施方式是提取水样图像中央 101×101 像素的图像。设原始图像的大小是 $M \times N$ ，则截取宽从第 $M/2 - 50$ 个像素点到第 $M/2 + 50$ 个像素点，高从第 $N/2 - 50$ 个像素点到第 $N/2 + 50$ 个像素点的子图像。
- 使用Python编程软件进行编程，即可把下图中左边的切割前的水样图像切割并保存到右边的切割后的水样图像。



分析方法与过程



```
for i in range(n):
```

```
    img = Image.open(path+imgNames[i]) # 读取图片
```

```
    M,N = img.size # 图片像素的尺寸
```

```
    img = img.crop((M//2-50,N//2-50,M//2+50,N//2+50)) # 图片切割
```

```
end
```

说明:

`img = Image.open(path)`, 这是PIL中的一个处理图片的函数, 使用时需

```
from PIL import Image
```

`img.crop` 返回图像的矩形区域, 参数为 (left, upper, right, lower)的元组

批量处理图片数据

imgNames = getImgNames() # 获取所有图片名称

n = len(imgNames) # 图片张数

for i in range(n):

img = Image.open(DEFAULT_PATH+imgNames[i]) # 读取图片

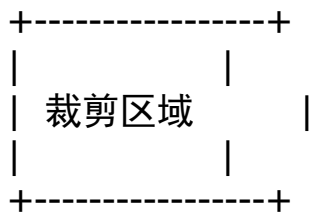
M,N = img.size # 图片像素的尺寸

img = img.crop((M//2-50,N//2-50,M//2+50,N//2+50)) # 图片切割

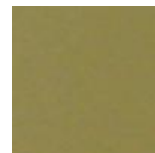
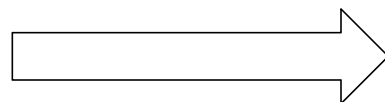
(0,0) → x



y (left, upper) (right, upper)



(left, lower) (right, lower)



分析方法与过程

第二步：特征提取

- 通过拍摄水样，采集得到水样图像，而图像数据的维度过大，不容易分析，需要从中提取水样图像的特征，提取反映图像本质的一些关键指标，以达到自动进行图像识别或分类的目的。显然，图像特征提取是图像识别或分类的关键步骤，图像特征提取的效果如何直接影响到图像识别和分类的好坏。
 - 图像特征主要包括有颜色特征、纹理特征、形状特征、空间关系特征等。与几何特征相比，颜色特征更为稳健，对于物体的大小和方向均不敏感，表现出较强的鲁棒性。本案例中由于水色图像是均匀的，故主要关注颜色特征。
 - 颜色特征是一种全局特征，描述了图像或图像区域所对应的景物的表面性质。一般颜色特征是基于像素点的特征，所有属于图像或图像区域的像素都有各自的贡献。在利用图像的颜色信息进行图像处理、识别、分类的研究中，在实现方法上已有大量的研究成果，主要采用颜色处理常用的直方图法和颜色矩方法等。

分析方法与过程

- 颜色直方图是最基本的颜色特征表示方法，它反映的是图像中颜色的组成分布，即出现了哪些颜色以及各种颜色出现的概率。其优点在于它能简单描述一幅图像中颜色的全局分布，即不同色彩在整幅图像中所占的比例，特别适用于描述那些难以自动分割的图像和不需要考虑物体空间位置的图像。其缺点在于它无法描述图像中颜色的局部分布及每种色彩所处的空间位置，即无法描述图像中的某一具体的对象或物体。
- 基于颜色矩提取图像特征的数学基础在于图像中任何的颜色分布均可以用它的矩来表示。根据概率论的理论，随机变量的概率分布可以由其各阶矩唯一的表示和描述。一副图像的色彩分布也可认为是一种概率分布，那么图像可以由其各阶矩来描述。**颜色矩包含各个颜色通道的一阶矩、二阶矩和三阶矩，对于一副RGB颜色空间的图像，具有R、G和B三个颜色通道，则有9个分量。**

分析方法与过程

第二步：特征提取

- 选择采用颜色矩来提取水样图像的特征，下面给出各阶颜色矩计算公式：

(1) **一阶颜色矩**：一阶颜色矩实际上就是颜色分布的均值 (Mean)，它反映了图像的整体明暗程度和颜色分布的平均水平。对于每个颜色通道，一阶颜色矩的计算公式为：

$$\mu_c = \frac{1}{N} \sum_{i=1}^N p_{c,i}$$

- μ_c : 第 c 个颜色通道的一阶矩 (均值)
- N : 图像中的像素总数
- $p_{c,i}$: 第 i 个像素在第 c 个颜色通道的值
- c : 颜色通道 (如 R, G, B 或 H, S, V)

```
data = np.zeros([n, 9]) # 用来装样本自变量  
labels = np.zeros([n]) # 用来放样本标签
```

```
for i in range(n):  
    img = Image.open(DEFAULT_PATH+imgNames[i]) # 读取图片  
    M,N = img.size # 图片像素的尺寸  
    img = img.crop((M//2-50,N//2-50,M//2+50,N//2+50)) # 图片切割
```

```
r,g,b = img.split() # 将图片分割成三通道
```

```
rd = np.asarray(r)/255 # 转化成数组数据
```

```
gd = np.asarray(g)/255
```

```
bd = np.asarray(b)/255
```

```
data[i,0] = rd.mean() # 一阶颜色矩
```

```
data[i,1] = gd.mean()
```

```
data[i,2] = bd.mean()
```

分析方法与过程

第二步：特征提取

(2) 二阶颜色矩

二阶颜色矩实际上是颜色分布的方差 (Variance)，它反映了颜色值的离散程度和分布范围。对于每个颜色通道，二阶颜色矩的计算公式为：

$$\sigma_c = \sqrt{\frac{1}{N} \sum_{i=1}^N (p_{c,i} - \mu_c)^2}$$

- σ_c : 第 c 个颜色通道的二阶颜色矩 (标准差)
- N : 图像中的像素总数
- $p_{c,i}$: 第 i 个像素在第 c 个颜色通道的值
- μ_c : 第 c 个颜色通道的一阶矩 (均值)
- c : 颜色通道 (如 R, G, B)

```
data = np.zeros([n, 9]) # 用来装样本自变量  
labels = np.zeros([n]) # 用来放样本标签
```

```
for i in range(n):
```

```
    img = Image.open(DEFAULT_PATH+imgNames[i]) # 读取图片  
    M,N = img.size # 图片像素的尺寸  
    img = img.crop((M//2-50,N//2-50,M//2+50,N//2+50)) # 图片切割
```

```
    r,g,b = img.split() # 将图片分割成三通道  
    rd = np.asarray(r)/255 # 转化成数组数据  
    gd = np.asarray(g)/255  
    bd = np.asarray(b)/255
```

```
    data[i,0] = rd.mean() # 一阶颜色矩  
    data[i,1] = gd.mean()  
    data[i,2] = bd.mean()
```

```
    data[i,3] = rd.std() # 二阶颜色矩
```

```
    data[i,4] = gd.std()
```

```
    data[i,5] = bd.std()
```

分析方法与过程

第二步：特征提取

(3) 三阶颜色矩

三阶颜色矩实际上是颜色分布的偏度 (Skewness)，它反映了颜色分布的不对称性。对于每个颜色通道，三阶颜色矩的计算公式为：

$$s_c = \sqrt[3]{\frac{1}{N} \sum_{i=1}^N (p_{c,i} - \mu_c)^3}$$

- s_c : 第 c 个颜色通道的三阶颜色矩
- N : 图像中的像素总数
- $p_{c,i}$: 第 i 个像素在第 c 个颜色通道的值
- μ_c : 第 c 个颜色通道的一阶矩 (均值)
- c : 颜色通道 (如 R, G, B)

```
# 自定义获取三阶颜色矩函数
```

```
def Skewness(data=None):
```

```
    x = np.mean((data-data.mean())**3)
```

```
    return np.sign(x)*abs(x)**(1/3)
```

```
data = np.zeros([n, 9]) # 用来装样本自变量
```

```
labels = np.zeros([n]) # 用来放样本标签
```

```
for i in range(n):
```

```
    ○ ○ ○ ○
```

```
    ○ ○ ○ ○
```

```
data[i,6] = Skewness(rd) # 三阶颜色矩
```

```
data[i,7] = Skewness(gd)
```

```
data[i,8] = Skewness(bd)
```

```
labels[i] = imgNames[i][0] # 样本标签
```



 3_21.jpg

分析方法与过程

对切割后的图像提取其颜色矩，作为图像的颜色特征。在颜色矩的提取中，提取每个文件名中的类别和序号，同时针对所有的图片执行相同的操作。得到下表。

表9-2 水色图像特征与相应的水色类别的部分数据

水质类别	序号	R 通道一阶矩	G 通道一阶矩	B 通道一阶矩	R 通道二阶矩	G 通道二阶矩	B 通道二阶矩	R 通道三阶矩	G 通道三阶矩	B 通道三阶矩
1	1	0.582 823	0.543 774	0.252 829	0.014 192	0.016 144	0.041 075	-0.012 64	-0.016 09	-0.041 54
2	1	0.495 169	0.539 358	0.416 124	0.011 314	0.009 811	0.014 751	0.015 367	0.016 01	0.019 748
3	1	0.510 911	0.489 695	0.186 255	0.012 417	0.010 816	0.011 644	-0.007 47	-0.007 68	-0.005 09
4	1	0.420 351	0.436 173	0.167 221	0.011 22	0.007 195	0.010 565	-0.006 28	0.003 173	-0.007 29
5	1	0.211 567	0.335 537	0.111 969	0.012 056	0.013 296	0.008 38	0.007 305	0.007 503	0.003 65
1	2	0.563 773	0.534 851	0.271 672	0.009 723	0.007 856	0.011 873	-0.005 13	0.003 032	-0.005 47
2	2	0.465 186	0.508 643	0.361 016	0.013 753	0.012 709	0.019 557	0.022 785	0.022 329	0.031 616
3	2	0.533 052	0.506 734	0.185 972	0.011 104	0.007 902	0.012 65	0.004 797	-0.002 9	0.004 214
4	2	0.398 801	0.425 56	0.191 341	0.014 424	0.010 462	0.015 47	0.009 207	0.006 471	0.006 764
5	2	0.298 194	0.427 725	0.097 936	0.014 778	0.012 456	0.008 322	0.008 51	0.006 117	0.003 47
1	3	0.630 328	0.594 269	0.298 577	0.007 731	0.005 877	0.010 148	0.003 447	-0.003 45	-0.006 53
2	3	0.491 916	0.546 367	0.425 871	0.010 344	0.008 293	0.012 26	0.009 285	0.009 663	0.011 549
3	3	0.559 437	0.522 702	0.194 201	0.012 478	0.007 927	0.012 183	0.004 477	-0.003 41	-0.005 29
4	3	0.402 068	0.431 443	0.177 364	0.010 554	0.007 287	0.010 748	0.006 261	-0.003 41	0.006 419
5	3	0.408 963	0.486 953	0.178 113	0.012 662	0.009 752	0.014 497	-0.006 72	0.002 168	0.009 992
1	4	0.638 606	0.619 26	0.319 711	0.008 125	0.006 045	0.009 746	-0.004 87	0.003 083	-0.004 5

分析方法与过程

模型构建

- 抽取80%作为训练样本，剩下的20%作为测试样本，用于水质评价检验，使用决策树算法构建水质评价模型。

```
from sklearn.model_selection import train_test_split
```

```
# 数据拆分,训练集、测试集
```

```
data_tr,data_te,label_tr,label_te = train_test_split(data,labels,test_size=0.2,  
random_state=10)
```

1. 数据拆分函数 `train_test_split`

```
from sklearn.model_selection import train_test_split
data_tr, data_te, label_tr, label_te = train_test_split(
    data,
    labels,
    test_size=0.2,
    random_state=10
)
```

随机数种子，应用于分割前对数据的洗牌。

`random_state` 设成定值意味着对于同一个数据集，只有第一次运行是随机的，随后多次分割只要 `random_state` 相同，则划分结果也相同。

参数解释：

`data`: 特征数据（自变量），形状为 `[n_samples, n_features]`

`labels`: 标签数据（因变量），形状为 `[n_samples]`

`test_size=0.2`: 测试集比例，0.2表示20%的数据作为测试集

`random_state=10`: 随机种子，确保每次拆分结果一致

返回值：

`data_tr`: 训练集特征数据（80%）

`data_te`: 测试集特征数据（20%）

`label_tr`: 训练集标签数据（80%）

`label_te`: 测试集标签数据（20%）

决策树由三类节点组成：

根节点：树的起点，包含所有训练数据

内部节点：表示一个特征测试，根据测试结果将数据分到不同分支

叶节点：表示最终的类别或预测值

决定今天是否适合打网球可能会考虑以下几个因素：

天气状况（晴朗、多云、雨天）

温度（高、中、低）

湿度（高、正常）

风力（强、弱）

决策过程可能如下：

如果天气是雨天且风力强 → 不打网球

如果天气是雨天但风力弱 → 再看湿度...

如果天气是晴朗且湿度高 → 不打网球

如果天气是晴朗且湿度正常 → 打网球

如果天气是多云 → 直接打网球

这种逐步判断的过程，正是决策树的工作方式。

收集了14天的历史数据，目标：根据天气、温度、湿度、风力这四个特征，构建一个模型来预测目标变量玩网球（是/否）

日期	天气 Outlook	温度 Temperature	湿度 Humidity	风力 Windy	玩网球 Play?
1	晴	高	高	弱	否
2	晴	高	高	强	否
3	多云	高	高	弱	是
4	雨	中	高	弱	是
5	雨	低	正常	弱	是
6	雨	低	正常	强	否
7	多云	低	正常	强	是
8	晴	中	高	弱	否
9	晴	低	正常	弱	是
10	雨	中	正常	弱	是
11	晴	中	正常	强	是
12	多云	中	高	强	是
13	多云	高	正常	弱	是
14	雨	中	高	强	否

第二步：选择根节点（第一次分裂）

决策树的核心是：选择哪个特征进行分裂，能最有效地“纯净”数据。原始数据集有9个“是”和5个“否”，比较混乱。

1. “天气”特征：

它有3个取值：晴、多云、雨。

- 当 *Outlook*=晴：样本有5天。其中，玩网球=是的有2天（第9，11天），玩网球=否的有3天（第1，2，8天）。纯净度很差。
- 当 *Outlook*=多云：样本有4天。其中，玩网球=是的有4天（第3，7，12，13天），玩网球=否的有0天。非常纯净！
- 当 *Outlook*=雨：样本有5天。其中，玩网球=是的有3天（第4，5，10天），玩网球=否的有2天（第6，14天）。纯净度一般。

“天气”一出手，就把“多云”这个分支完全分对了（全是“是”）。

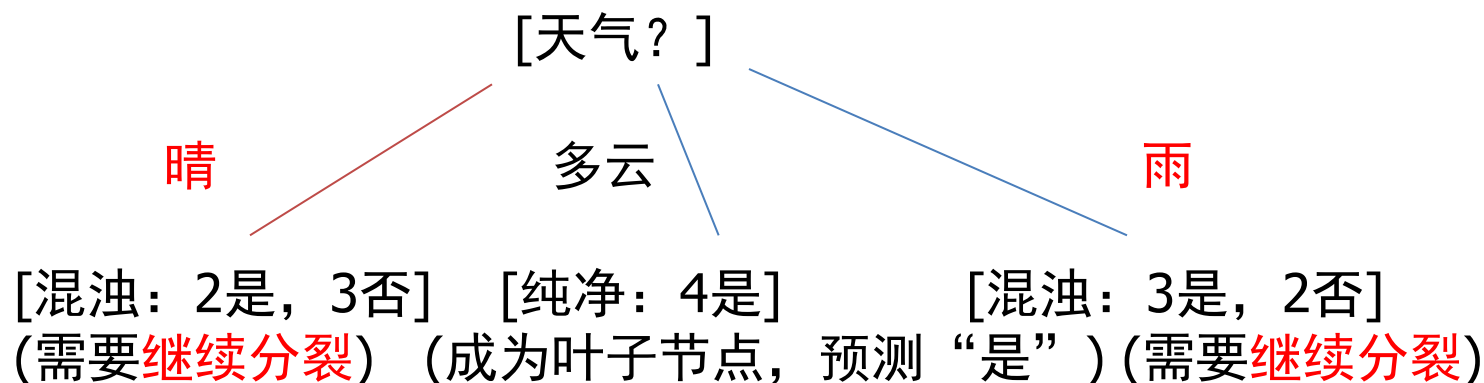
2. 其他特征，比如“湿度”：

它有2个取值：高、正常。

- 当 Humidity=高：样本有7天。其中，“是”有3天，“否”有4天。

- 当 Humidity=正常：样本有7天。其中，“是”有6天，“否”有1天。

虽然“正常”这边也比较纯净，但不如“天气”特征中的“多云”分支（100%纯净）效果好。经过计算（信息增益或基尼系数），会发现**“天气”是区分度最高的特征**。因此选择它作为**根节点**。



1. 天气=晴 这个分支（5个样本）

在这个分支里只看天气是晴的数据（第1, 2, 8, 9, 11天）。目标是在这个子集上找到最好的分裂特征。

•看“湿度”：

- Humidity=高：3个样本，全是“否”（第1, 2, 8天）。非常纯净！
- Humidity=正常：2个样本，全是“是”（第9, 11天）。非常纯净！
- 用“湿度”把这个分支分干净了。

所以，对于“晴”分支，选择“湿度”作为分裂节点。

2. 天气=雨 这个分支（5个样本）

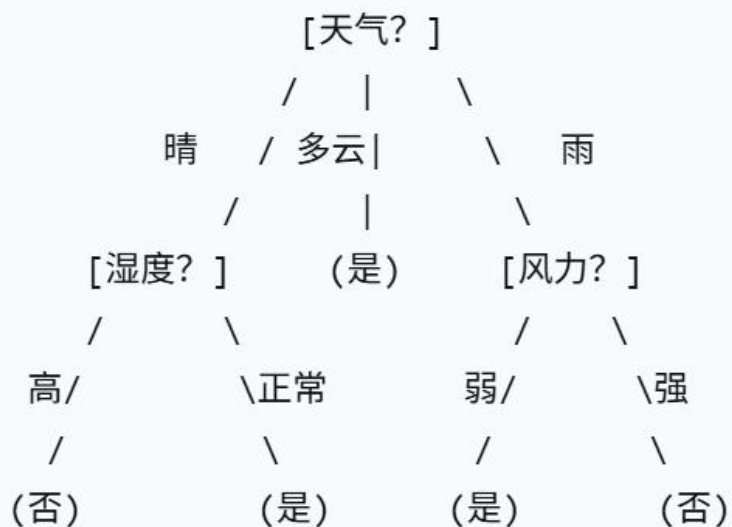
在这个分支里只看天气是雨的数据（第4, 5, 6, 10, 14天）。

•看“风力”：

- Windy=弱：3个样本，全是“是”（第4, 5, 10天）。非常纯净！
- Windy=强：2个样本，全是“否”（第6, 14天）。非常纯净！
- 用“风力”把这个分支分干净了。

所以，对于“雨”分支，选择“风力”作为分裂节点。

使用决策树进行预测



例子1： 新的一天天气是 多云。

从根节点开始：天气 -> 多云 -> 直接到达叶子节点 “是”。

预测结果：去玩。

例子2： 新的一天天气是 晴，湿度是高

从根节点开始：天气 -> 晴 -> 走到 “湿度？” 节点 -> 湿度=高 -> 到达叶子节点 “否”。

预测结果：不去玩。

例子3： 新的一天天气是 雨，风力是强

从根节点开始：天气 -> 雨 -> 走到 “风力？” 节点 -> 风力=强 -> 到达叶子节点 “否”。

预测结果：不去玩。

分析方法与过程

模型训练

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt_classifier = DecisionTreeClassifier(random_state=5).fit(data_tr,  
label_tr)
```

```
random_state=5
```

控制决策树的随机性，确保每次运行结果一致，便于结果复现和调试

返回值是：一个**DecisionTreeClassifier**对象，将其赋值给变量 dt_classifier。

创建决策树模型

```
dt_classifier = DecisionTreeClassifier(
```

```
    random_state=5,          # 随机种子
```

```
    max_depth=5,             # 树的最大深度
```

```
    min_samples_split=10,    # 内部节点再划分所需最小样本数
```

```
    min_samples_leaf=5,      # 叶子节点最少样本数
```

```
    criterion='gini'         # 分裂标准
```

```
)
```

关键参数详解：

random_state=5

- 控制决策树的随机性，确保每次运行结果一致，便于结果复现和调试

max_depth=5

- 决策树的最大深度

- 值越小：树很“浅”，模型简单。它只能使用很少的特征和简单的规则来做决策。模型越简单，可能欠拟合。

- 值越大：树很“深”，模型复杂。它可以利用更多的特征、更复杂的组合和更精细的规则来划分数据。模型越复杂，可能过拟合

- 最佳的最大深度**不是一个理论值**，而是需要通过实验来确定的无论是过浅还是过深，都会损害模型的**泛化能力**(是指模型在未知数据上的表现)。

min_samples_split=10, # 内部节点再划分所需最小样本数

min_samples_split 规定了一个决策树的内部节点（非叶子节点）必须至少拥有多少个样本，才有资格被继续划分（分裂）。

内部节点：树中那些继续进行条件判断的节点。

划分/分裂：根据某个特征和阈值将当前节点的数据分成两个或多个子集的过程。

所以，min_samples_split=10 的直接含义是：**如果一个节点中的样本数少于10个，这个节点将不再继续分裂，而是直接成为一个叶子节点。**

**max_depth 是从“纵向”控制深度，
而 min_samples_split 是从“横向”
或“局部”控制**

`min_samples_leaf=5` 它从结果的角度来控制树的生长

`min_samples_leaf` 规定了一个决策树的叶子节点（终端节点）必须包含的最少样本数。

叶子节点：树中最末端的节点，不再进行任何划分，用于做出最终的预测（分类或回归）。

`min_samples_leaf=5` 的直接含义是：任意一个叶子节点中，都必须至少包含5个样本。

它强迫每个预测（叶子节点）都必须基于一定数量的样本（至少5个）的“集体决策”。在分类中，叶子节点的类别由该节点内样本的多数票决定

三个决策树最常用的**预剪枝参数**，它们从不同角度限制树的生长：

1.max_depth：从根节点到叶子节点的最大“步数”。（**纵向限制**）

2.min_samples_split：一个节点**能够被分裂**所需的最小样本数。（**分裂资格限制**）

3.min_samples_leaf：一个节点**分裂后**，其每个子节点（叶子节点）必须包含的最小样本数。（**分裂结果限制**）

它们的关系：

一个节点要发生分裂，必须同时满足：

- 当前深度 $< \text{max_depth}$
- 当前节点样本数 $\geq \text{min_samples_split}$
- 并且分裂后，产生的**左右两个子节点**的样本数都 $\geq \text{min_samples_leaf}$

创建决策树模型

dt_classifier = DecisionTreeClassifier(

random_state=5, *# 随机种子*

max_depth=5, *# 树的最大深度*

min_samples_split=10, *# 内部节点再划分所需最小样本数*

min_samples_leaf=5, *# 叶子节点最少样本数*

criterion='gini' *# 分裂标准*

)

`criterion='gini'` 指使用**基尼指数（基尼不纯度）**作为分裂标准。

基尼不纯度衡量的是：从节点中**随机抽取两个样本，它们属于不同类别的概率**。

基尼不纯度越低，说明节点的纯度越高（即节点中大部分样本都属于同一个类别）。

基尼不纯度越高，说明节点的混乱度越高（即节点中样本的类别分布很均匀）。

对于一个给定的节点 t ，包含 K 个类别，基尼不纯度的计算公式为：

$$Gini(t) = 1 - \sum_{i=1}^K (p_i)^2$$

p_i 是节点 t 中第 i 类样本所占的比例。

假设一个节点中有10个样本：

情况A（纯）：10个样本全属于第1类。

- $p_1 = 1, p_2 = 0$
- $Gini = 1 - (1^2 + 0^2) = 0 \rightarrow$ 完全纯净，理想状态

情况B（均匀混合）：5个第1类，5个第2类。

- $p_1 = 0.5, p_2 = 0.5$
- $Gini = 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = 0.5 \rightarrow$ 非常不纯，最混乱状态

情况C（比较纯）：8个第1类，2个第2类。

- $p_1 = 0.8, p_2 = 0.2$
- $Gini = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 0.32 \rightarrow$ 比较纯净

决策树通过计算基尼不纯度的减少量（即“基尼增益”）来选择最佳分裂。

基尼增益的计算公式：

$$\Delta Gini = Gini(parent) - \sum_{child=1}^2 \frac{N_{child}}{N_{parent}} \cdot Gini(child)$$

1. 计算父节点的基尼不纯度。
 2. 对于每个候选特征和阈值，计算分裂后产生的左子节点和右子节点的基尼不纯度。
 3. 计算这两个子节点基尼不纯度的加权平均（权重为子节点样本数占总样本数的比例）。
 4. 用父节点的基尼不纯度减去这个加权平均值，得到基尼增益。
 5. **选择基尼增益最大**的那个分裂方案。
- 树总是在**寻找那个能让子节点“纯度”提升最多的分裂方式**。

分析方法与过程

水质评价

```
from sklearn.metrics import confusion_matrix
pre_te = dt_classifier.predict(data_te)
```

混淆矩阵

```
cm_te = confusion_matrix(label_te, pre_te)
print(cm_te)
```

```
from sklearn.metrics import accuracy_score
```

准确率

```
print(accuracy_score(label_te, pre_te))
```

实际\预测	预测为0	预测为1
实际为0	TN	FP
实际为1	FN	TP

• **TN (True Negative)**: 实际为负类，预测为负类 ✓

• **FP (False Positive)**: 实际为负类，预测为正类 ✗ (误报)

• **FN (False Negative)**: 实际为正类，预测为负类 ✗ (漏报)

• **TP (True Positive)**: 实际为正类，预测为正类 ✓

		预测类别				
		苹果	香蕉	橙子	葡萄	芒果
真实类别	苹果	[[3, 1, 0, 0, 0]				
	香蕉	[0, 3, 1, 0, 0]				
	橙子	[0, 0, 2, 1, 0]				
	葡萄	[0, 0, 1, 2, 0]				
	芒果	[0, 0, 0, 1, 3]]				

正确分类（对角线）：

- 苹果：3个正确 ✓
- 香蕉：3个正确 ✓
- 橙子：2个正确 ✓
- 葡萄：2个正确 ✓
- 芒果：3个正确 ✓

错误分类分析：

- 苹果→香蕉：1个 ✗
- 香蕉→橙子：1个 ✗
- 橙子→葡萄：1个 ✗
- 葡萄→橙子：1个 ✗
- 芒果→葡萄：1个 ✗

203

```
[[ 6  1  2  0  0]
 [ 0  8  0  0  0]
 [ 1  0 11  1  0]
 [ 0  0  4  3  2]
 [ 1  0  0  0  1]]|
```

0.7073170731707317

总样本数：203 测试 $203 \times 0.2 = 41$

混淆矩阵：

```
[[ 6  1  2  0  0]
 [ 0  8  0  0  0]
 [ 1  0 11  1  0]
 [ 0  0  4  3  2]
 [ 1  0  0  0  1]]
```

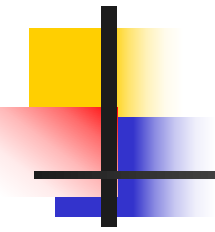
准确率： $(6+8+11+3+1) / 41 = 0.7073170731707317$

小结

基于水色图像特征提取的水质评价

- a) 预处理样本数据
- b) 把数据随机分为两部分，一部分用于训练，一部分用于测试。
- c) 使用scikit-learn里的DecisionTreeClassifier函数以及训练数据构建决策树模型，使用predict函数和构建的决策树模型分别对训练数据进行分类，使用scikit-Learn的子库metrics的confusion_matrix函数求出混淆矩阵，如果仅仅是想知道准确率，可以用metrics的accuracy_score函数返回。
- d) 使用predict函数和步骤（3）构建好的决策树模型分别对测试数据进行分类，参考步骤（3）得到模型分类正确率和混淆矩阵。





Any questions?